OPERATIONS ANALYTICS / INDEPENDENT CASE STUDY

Smart Warehouse Intelligence

Technical project report

Ankith Ramesh Babu

26 October 2026 | Release 2

Inventory reconciliation, demand fulfillment and replenishment planning.

This report documents a reproducible analysis of synthetic operational records. It connects the business questions, calculation rules, software implementation, dashboard views and verification evidence in one reviewable package.

Scope	Default dataset	Primary outputs
Four warehouse regions	5,760 daily observations	Python analysis, HTML dashboards, Power BI project source, CSV exports

Results describe the included simulation. They are not employer performance measurements, deployed ERP outcomes or quantified cost savings.

Repository: github.com/ankithr457/smart-warehouse-intelligence

Executive summary and contents

The analysis separates reconciliation from replenishment. Daily ERP and WMS snapshots identify discrepancies, while a separate SKU-level decision table uses recent demand to propose replenishment quantities. A discrepancy is an investigation signal, not evidence of physical loss.

Result	Value	Interpretation
Discrepancies	2,245 / 5,760	38.98% of observations
Demand fulfillment	99.56%	69,327 / 69,633 demanded units
Replenishment	469 units	Two proposals at the final snapshot

Report navigation

Section	Page
Business scope and architecture	3
Data specification and provenance	4
Calculation methodology	5
Results and operational interpretation	6
Dashboard overview and interaction	7-8
Power BI model and report layout	9
Sensitivity analysis	10
Implementation excerpts	11
Validation and reproducibility	12
Limitations and conclusion	13
References and artifact index	14

Business scope and architecture

Questions addressed

- Where do ERP and WMS snapshots disagree?
- How much demand remains unfulfilled?
- Which warehouse-SKU pairs cross the reorder threshold?
- How sensitive are proposals to assumed lead time?

Processing sequence

Stage	Implementation	Output
Generate	Seeded Python simulation	Operational records
Calculate	Documented business rules	KPIs and decision fields
Export	Deterministic CSV / JSON writer	Auditable report files
Present	HTML and PBIR visual definitions	Interactive review surfaces
Verify	Unit tests, schema and browser checks	Validation evidence

Design decisions

The analysis uses Python standard-library functions to keep execution portable. The browser report embeds its data and needs no server. The Power BI model embeds the same CSV snapshot in Power Query, avoiding machine-specific paths. Data generation and decision rules remain separate from presentation.

Operational boundaries

This is a batch analysis of simulated observations. It does not write to SAP, execute procurement transactions, reserve stock or authorize operational changes. Each exported row retains the identifiers needed to trace a result back to its source observation.

Data specification and provenance

All source observations are generated in src/run.py with a fixed random seed. No customer, supplier or employee data is used. Regional names represent four analytical groups, not actual sites.

Field	Type	Meaning
warehouse + sku + day	Composite key	4 warehouses x 12 SKUs x 120 days
opening_stock	Integer	Previous ERP closing stock
received	Integer	Sampled from 0, 0, 0, 0, 30, 50
demand	Integer	Daily demand sampled from 2 to 22
shipped	Integer	min(opening stock + receipts, demand)
erp_stock	Integer	Opening + received - shipped
wms_stock	Integer	max(0, ERP stock + sampled noise)
variance	Integer	WMS minus ERP stock

Each warehouse-SKU series starts with 120 to 500 ERP units. WMS noise is sampled from -3, 0, 0, 0, 2. Stock observations are snapshots, so totals across dates are not an inventory balance. Seed 43 produces the baseline. Simulation days are numbered periods, not calendar dates.

Traceability

Committed CSVs, KPI JSON, analysis JSON and the Power BI embedded snapshot refer to the same default run. Changes to seeds or assumptions require regenerated reports and a new result interpretation.

Calculation methodology

Calculation	Rule
History window	Latest 28 observations per warehouse-SKU
Demand variability	Sample standard deviation (n - 1)
Safety stock	1.65 x standard deviation x sqrt(lead time)
Reorder point	ceil(mean demand x lead time + safety stock)
Target stock	ceil(mean demand x 21 target days + safety stock)
Trigger	Latest WMS stock <= reorder point
Proposal	max(target stock - WMS stock, 0), only when triggered

Worked example

At mean demand 12 units/day, standard deviation 4 and lead time 7 days, the reorder point is ceil(84 + 17.46) = 102 units. The 21-day target is 270. Stock of 90 triggers a proposal for 180 units. Stock of 120 does not trigger an order, even though it is below target.

Decision timing

Proposals use the final day after closing stock is known. They do not feed back into the receipt simulation. This is a decision snapshot, not a tested replenishment policy. The z value is illustrative and does not establish achieved service probability.

Results and interpretation

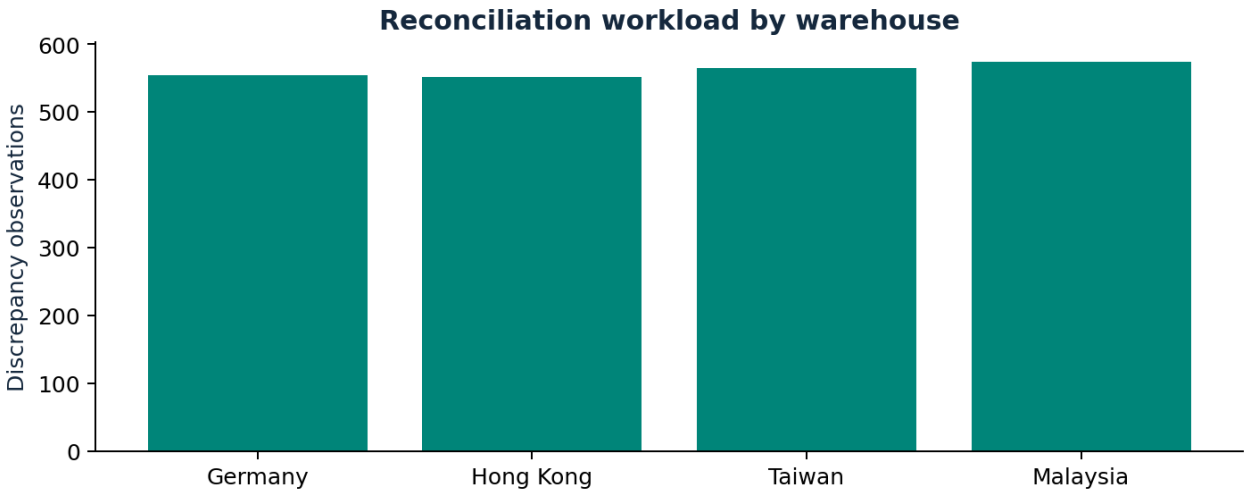


Figure 1. Baseline results calculated from the committed operational records.

Warehouse	Discrepancies	Demand	Shipped
Germany	554	17,523	17,371
Hong Kong	552	17,470	17,316
Taiwan	565	17,444	17,444
Malaysia	574	17,196	17,196

The dataset contains 306 unfulfilled demand units. High aggregate fulfillment does not remove SKU-level risk: Germany SKU005 ends at zero WMS stock and receives a 290-unit proposal. Germany SKU002 receives 179 units. Both final proposal rows have zero ERP/WMS variance.

The 2,245 discrepancy count describes noisy observations across time, not 2,245 distinct damaged items or stock-loss events. Repeated observations of a SKU must not be interpreted as independent physical losses.

Interactive dashboard overview

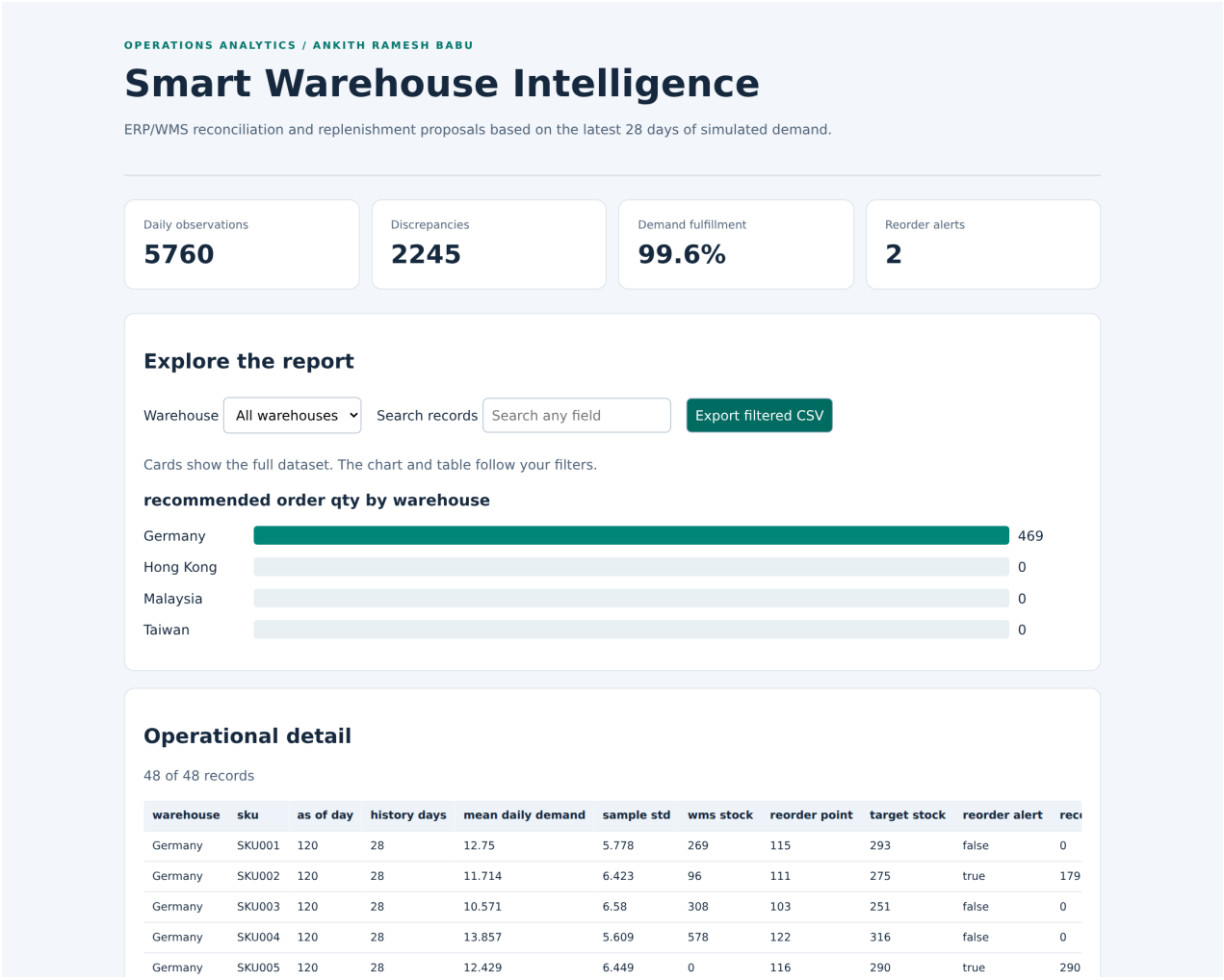


Figure 2. Actual browser capture of the generated HTML dashboard, baseline run.

The overview combines full-dataset KPI cards with warehouse filtering, text search, a comparative bar chart and a detailed register. Its data is embedded in the HTML, so the report opens offline without account access.

Display precision differs from the calculation exports: the HTML KPI cards show one decimal place for ratios, while KPI JSON preserves four decimal places. This is display rounding, not a calculation difference.

Dashboard interaction and audit trail

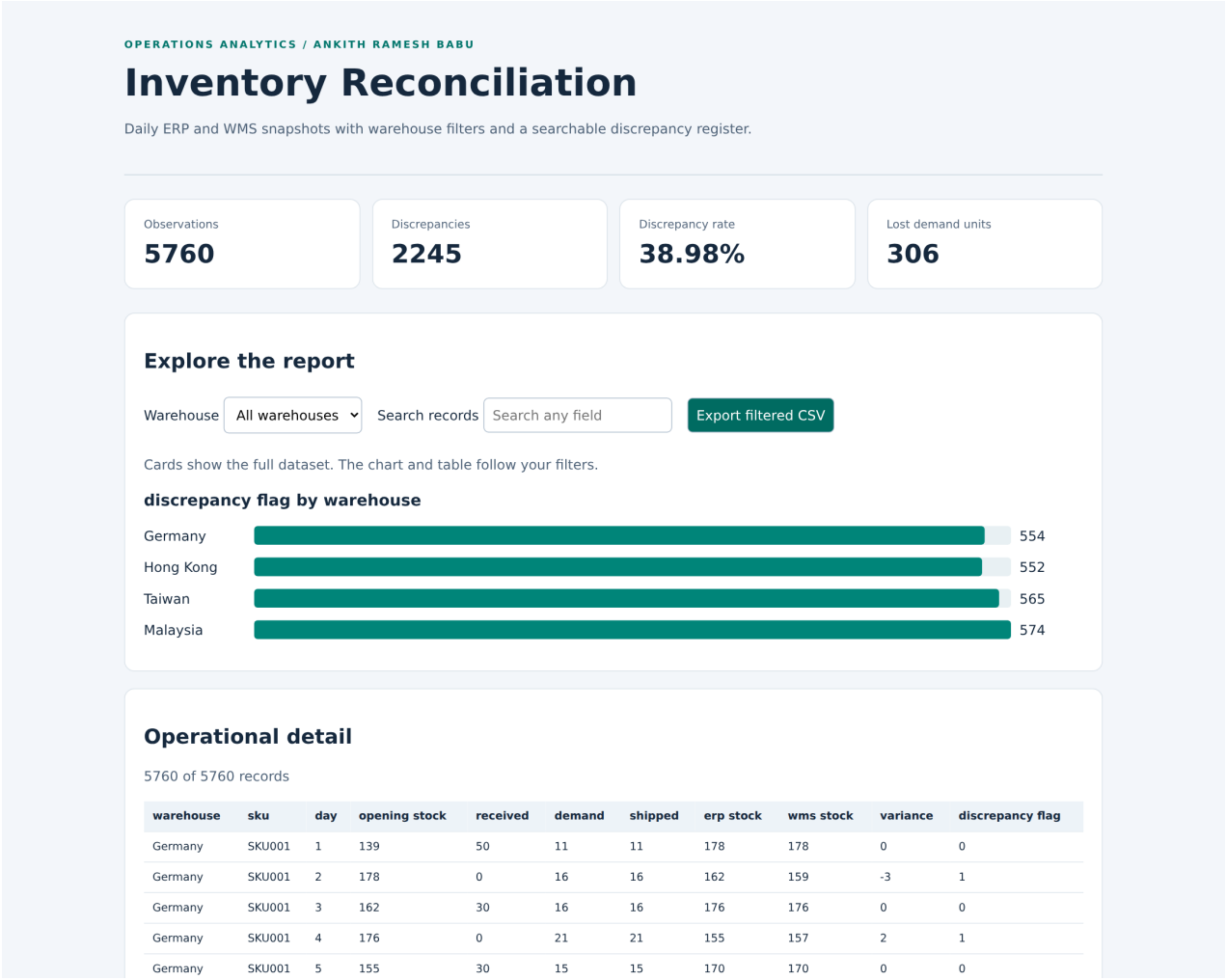


Figure 3. Actual browser capture of the daily inventory reconciliation view.

Warehouse and search filters update the chart and visible detail rows. Summary cards intentionally remain scoped to the complete dataset. The table displays at most 200 matching rows for responsiveness. CSV export includes every matching row, not only the displayed sample.

In the browser checks, selecting Germany returned records, an unmatched search returned zero rows, and filtered CSV download succeeded. The layout also passed a 390-pixel mobile-width overflow check. The wide detail table remains horizontally scrollable.

Power BI model and page layout

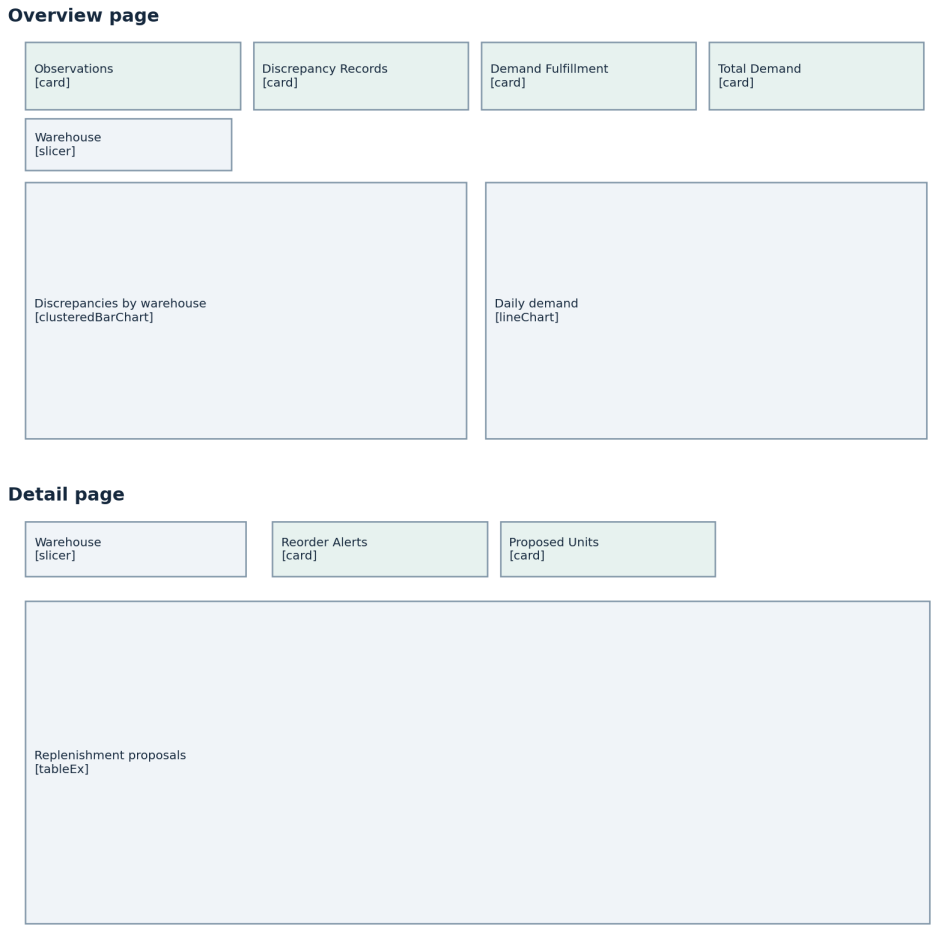


Figure 4. Layout diagram derived from PBIR visual coordinates. This is not a Power BI Desktop screenshot.

The package includes a PBIP entry point, PBIR report pages and a TMSL model.bim semantic model. Power Query decodes an embedded CSV snapshot, promotes headers and applies explicit data types. DAX measures provide aggregation under report filter context.

The overview page groups KPI cards, a warehouse slicer and two charts. The detail page exposes replenishment proposals and a separate warehouse slicer. The daily and recommendation tables are independent grains, so each page uses measures from its own table.

Verification boundary: JSON schema and field-reference checks cover the source package. Power BI Desktop rendering, refresh and DAX execution remain unverified because the Windows application is unavailable in this environment. No PBIX or Desktop screenshot is represented as tested.

Sensitivity and comparative analysis

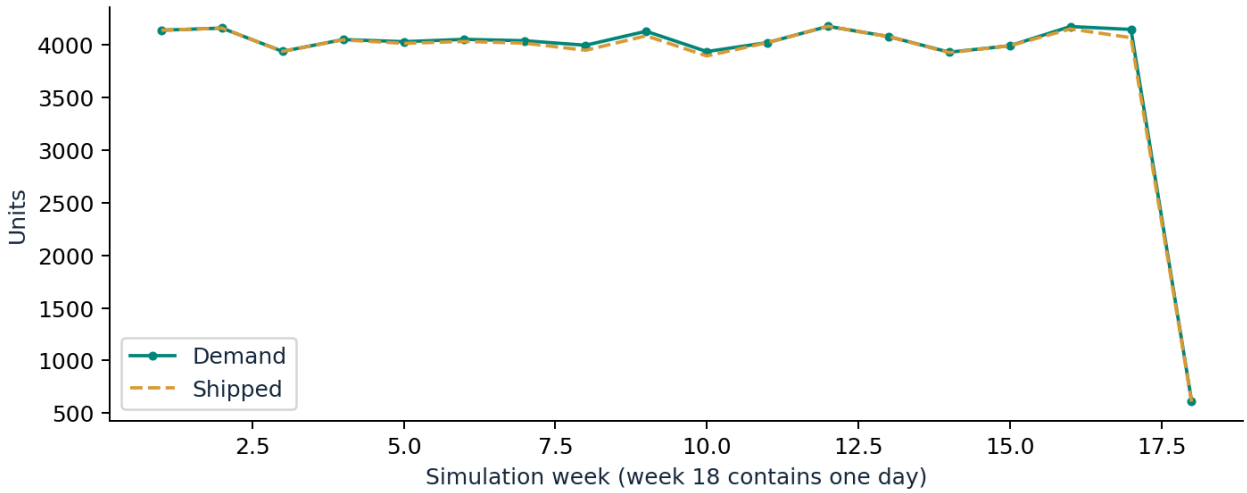


Figure 5. Weekly demand and shipped units. The final period contains only day 120.

Lead time (days)	Reorder alerts	Proposed units
3	1	280
7	2	469
10	2	479
14	5	836

With the same closing stock and demand history, increasing lead time from 3 to 14 days increases the alert count from one to five and proposed units from 280 to 836. The 21-day target horizon remains fixed. This isolates the effect of lead-time demand and safety stock.

No service-level improvement is estimated from these proposals because they are not simulated as future receipts. A policy comparison would require a forward simulation with supplier lead-time variation, purchase orders and holding-cost assumptions.

Implementation excerpts

Reorder threshold

```
def reorder_point(mean_daily_demand, daily_std,
                  lead_time=7, z=1.65):
    if min(mean_daily_demand, daily_std, lead_time, z) < 0:
        raise ValueError("Inputs must be nonnegative")
    return math.ceil(
        mean_daily_demand * lead_time
        + z * daily_std * math.sqrt(lead_time)
    )
```

Equivalent formatted excerpt of src/run.py. The production function uses the same calculation and rejects negative inputs. The recommendation function adds history selection, target stock and the trigger condition.

Stock conservation

```
shipped = min(erp_stock + received, demand)
erp_stock += received - shipped
wms_stock = max(0, erp_stock + measurement_noise)
```

The last line illustrates the WMS observation rule. Measurement noise does not change ERP closing stock and is not carried forward as a physical adjustment.

Code organization

src/run.py owns simulation and business rules. src/reporting.py handles CSV and HTML rendering. scripts/analyze.py creates scenario evidence. scripts/build_powerbi.py regenerates the embedded snapshot and report definitions. Tests exercise calculations, edge cases and report generation.

Validation and reproducibility

Check	Evidence	Outcome
Business-rule tests	Stock conservation, reorder examples and invalid inputs	Passed
Repeat generation	Same seed and arguments reproduce identical exports	Passed
Browser interactions	Warehouse filter, empty search, CSV export, mobile width	Passed
Report source	PBIR JSON schema and model-field linkage checks	Passed
Desktop Power BI	Windows application, refresh and DAX engine	Not executed

Reproduction commands

```
python src/run.py
python scripts/analyze.py
python scripts/build_powerbi.py
python -m unittest discover -s tests -v
```

The default analysis uses the Python standard library. Report authoring dependencies are isolated in requirements-report.txt. PDF charts and tables draw from committed CSV and JSON outputs. Browser screenshots record the generated HTML interface rather than a hand-drawn imitation.

Cross-checks

All 5,760 rows satisfy ERP stock conservation. Shipments never exceed demand or available ERP stock. The 48-row recommendation table contains one latest proposal per warehouse-SKU pair. Recommended quantity is zero whenever the trigger is false.

Scope of assurance

Passing these checks establishes internal consistency for the stated rules and dataset. It does not establish calibration against a real warehouse, production readiness or economic benefit.

Limitations and conclusion

Model limitations

• Lead time is fixed and daily demand is treated as independent.

• No open purchase orders, backorders, reservations, MOQ or expiry constraints are modeled.

• WMS variance is injected measurement noise, not an inferred reconciliation cause.

• Proposals are evaluated at one closing snapshot and not executed in the simulation.

Next engineering priorities

An operational extension should introduce validated external data, explicit business definitions, schema checks and exception ownership. A replenishment policy backtest should model order placement, supplier lead times, receipts and total inventory cost.

Conclusion

The project provides an auditable chain from simulated observations to decision fields, exports and visual review. Its main contribution is the separation of stock reconciliation from demand-based replenishment decisions.

The reported outputs are reproducible and the calculation boundaries are documented. Power BI source is supplied as an editable project, with Desktop verification explicitly separated from the automated source checks.

References and artifact index

[1] Python Software Foundation. random: Generate pseudo-random numbers.
https://docs.python.org/3/library/random.html

[2] Python Software Foundation. statistics: Mathematical statistics functions.
https://docs.python.org/3/library/statistics.html

[3] Microsoft. Power BI Desktop projects.
https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-overview

[4] Microsoft. Power BI project report folder and PBIR.
https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-report

[5] Microsoft. Power BI project semantic model folder.
https://learn.microsoft.com/en-us/power-bi/developer/projects/projects-dataset

[6] Microsoft. DIVIDE function (DAX).
https://learn.microsoft.com/en-us/dax/divide-function-dax

Sources consulted 26 October 2026. The inventory policy is an illustrative project assumption. Documentation references support software formats and function behavior, not a claim of real-world model validation.

Artifact index

Path	Purpose
src/	Simulation, calculations and HTML reporting
reports/*.csv, *.json	Baseline and scenario evidence
reports/dashboard.html	Interactive operational dashboard
reports/figures/	Actual browser captures, charts and layout diagram
powerbi/	PBIP, PBIR and semantic model source
tests/	Automated checks
docs/methodology.md	Definitions and assumptions
reports/project-report.pdf	This technical report